

CADENCE

System Architecture Document

Causal Attribution-Driven Efficient Continual Retraining for Production ML Systems

Version 1.0

Prepared by: Ankita

Document status: Draft — living document, updated as design decisions are finalized

Table of Contents

- 1. Purpose and Scope
- 2. System Overview
- 3. High-Level Architecture
- 4. Component Architecture
 - 4.1 Inference Gateway & Streaming Collector
 - 4.2 Drift Detection Trigger
 - 4.3 CDAG Builder (Causal Discovery)
 - 4.4 GNN (Graph Embedding)
 - 4.5 Counterfactual Surrogate
 - 4.6 PPO Retraining Scope Optimizer
 - 4.7 Retraining Executor (EWC / Full Retrain)
 - 4.8 Shadow / Canary Validator
 - 4.9 Supporting Services
- 5. Model Adapter Layer (Multi-Model-Type Support)
- 6. Data Flow
- 7. Deployment View
- 8. Technology Stack
- 9. Non-Functional Considerations
- 10. Open Design Decisions
- Appendix A: Glossary
- Appendix B: Revision History

1. Purpose and Scope

This document describes the system architecture of CADENCE (Causal Attribution-Driven Efficient Continual Retraining for Production ML Systems) — a closed-loop MLOps system that diagnoses the true cause of production ML model performance degradation and automatically selects the minimal sufficient retraining action, balancing compute cost, carbon cost, and service-level performance requirements.

This document is intended for engineering planning, implementation reference, and technical review (including academic/patent review). It defines every major component, the data contracts between components, the technology choices, and the design decisions that remain open.

Scope: this version of the architecture targets small-to-mid-sized structured-prediction models (classifiers, regressors, ranking models). Large Language Models (LLMs) are explicitly out of scope for this version and are addressed separately as a future-work extension.

2. System Overview

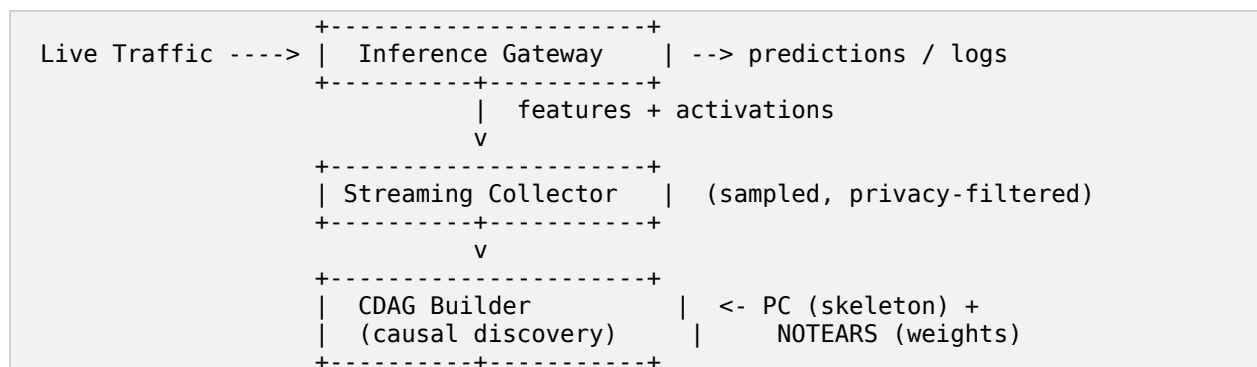
Production ML models degrade over time as real-world data drifts away from training-time distributions. Existing monitoring tools detect that drift occurred but cannot explain why, and most retraining pipelines either retrain on a fixed schedule (wasteful) or wait for a human to notice (slow and costly).

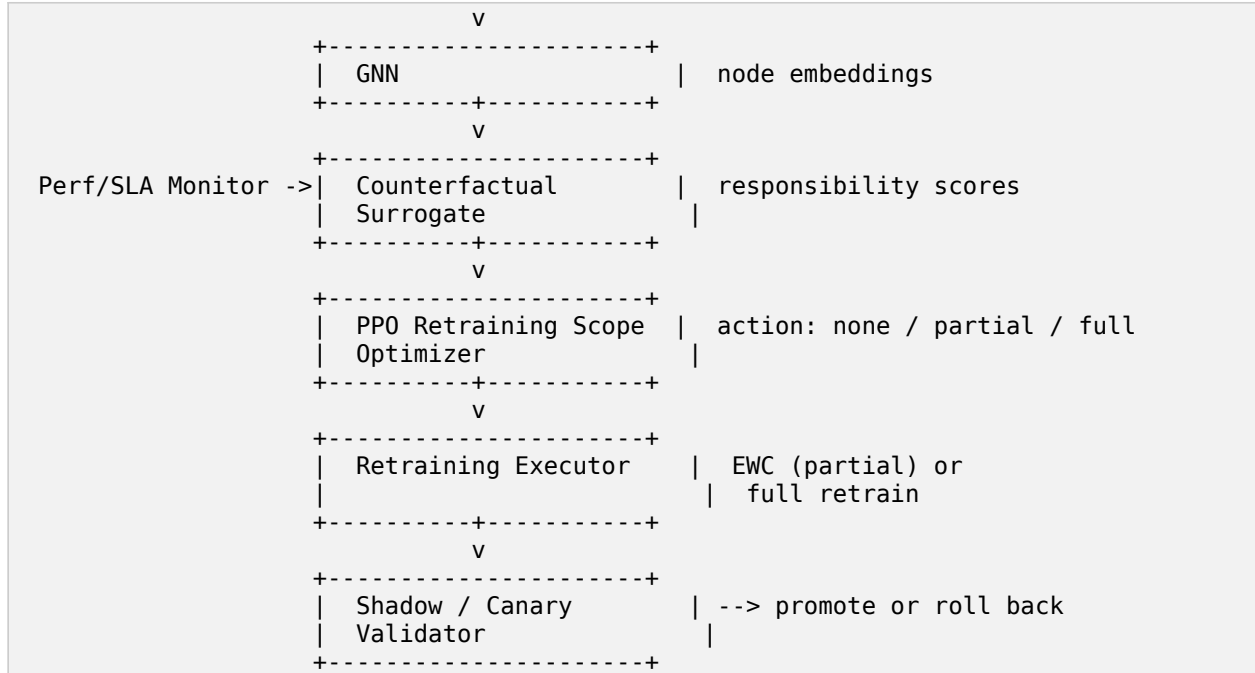
CADENCE closes this loop with three coordinated innovations, layered on top of the model being monitored:

- Causal source attribution — identifies which specific upstream factor (an external feature or an internal model representation) is responsible for a performance drop, rather than reporting an aggregate drift score.
- Calibrated, cost-aware retraining scope selection — a learned policy chooses between no action, a targeted partial fix, or a full retrain, based on predicted recovery, compute cost, and carbon cost.
- Continual-learning-safe execution — any partial fix is applied with forgetting-prevention safeguards, and every change is validated in shadow/canary mode before reaching real traffic.

3. High-Level Architecture

The diagram below shows the major components and the direction of data flow between them.





Supporting services (not shown above): model registry, cost/carbon estimator, monitoring dashboard, and an audit log for compliance and explainability.

4. Component Architecture

Each component below is described by its purpose, inputs, outputs, and the algorithm/technology it uses.

4.1 Inference Gateway & Streaming Collector

Field	Description
Purpose	Serve live predictions and capture the telemetry CADENCE needs, without disrupting production latency
Input	Live feature vectors from real traffic
Output	Predictions (to caller) + logged tuples: (features, prediction, internal activations, delayed label if available)
Technology	FastAPI / Triton Inference Server; forward hooks for activation capture; Kafka or Redis Streams for the logging path

4.2 Drift Detection Trigger

Field	Description
Purpose	Cheap, always-on monitor that decides when to invoke the more expensive CDAG/attribution pipeline
Input	Rolling windowed feature statistics; delayed-label performance when available
Output	A binary alert (fire / no fire) with the triggering window's metadata
Technology	Population Stability Index (PSI) and/or Kolmogorov-Smirnov test on rolling windows

4.3 CDAG Builder (Causal Discovery)

Field	Description
Purpose	Construct a causal graph connecting external feature drift, internal activation-cluster shifts, and the performance outcome
Input	Rolling windowed statistics for every feature node, activation-cluster node, and the performance node
Output	A weighted, directed graph (the CDAG): skeleton + edge weights
Technology	Hybrid: PC algorithm (conditional-independence testing) for the skeleton, NOTEARS-style continuous optimization for edge weights restricted to that skeleton

4.4 GNN (Graph Embedding)

Field	Description
Purpose	Produce a contextualized embedding per CDAG node, capturing multi-hop causal influence
Input	CDAG structure: per-node feature vectors, per-edge weights
Output	Per-node embedding vector
Technology	GraphSAGE / GAT-style graph neural network; trained jointly with the counterfactual surrogate

4.5 Counterfactual Surrogate

Field	Description
Purpose	Estimate, without performing a real intervention, what performance would result if a given node's cause were corrected
Input	Node embedding + current drift-magnitude context
Output	Predicted post-fix performance; subtracted from current performance to yield a responsibility score per node
Technology	MLP or gradient-boosted trees; trained offline on sandbox drift-injection-and-fix experiments plus historical incident outcomes

4.6 PPO Retraining Scope Optimizer

Field	Description
Purpose	Decide the minimal sufficient retraining action for the current situation
Input (state)	Responsibility scores, current performance vs. SLA, drift duration, per-action compute/carbon cost estimate, time since last retrain
Output (action)	One of: no-op, partial retrain, full retrain
Technology	PPO (Proximal Policy Optimization) with a Lagrangian-constrained reward (compute cost + carbon cost + SLA-violation penalty), trained in a simulated sandbox environment

4.7 Retraining Executor (EWC / Full Retrain)

Field	Description
Purpose	Apply the chosen fix to the production model
Input	The chosen action; for partial retrain, the identified responsible subnetwork/layer
Output	An updated (but not yet promoted) candidate model
Technology	Partial: freeze all but the targeted layer(s), fine-tune with an Elastic Weight

Field	Description
	Consolidation (EWC) penalty plus a replay buffer of historical data. Full: standard retrain of the entire model on historical + new data

4.8 Shadow / Canary Validator

Field	Description
Purpose	Confirm a candidate model actually recovers performance before it serves real traffic
Input	Candidate model + a held-out recent validation window
Output	Promote (full rollout via canary) or roll back (log failure, feed outcome back to the surrogate as a new training example)
Technology	Shadow-mode scoring in parallel with production; staged canary rollout on promotion

4.9 Supporting Services

- Model Registry — versioned storage of production model checkpoints and metadata (MLflow).
- Cost/Carbon Estimator — compute-hours x grid carbon-intensity, feeding the PPO reward (CodeCarbon / Electricity Maps API).
- Monitoring Dashboard — visualizes drift alerts, CDAG attributions, and retraining history (Grafana + Prometheus, or a custom app).
- Audit Log — records every decision and its justification for compliance and explainability review.

5. Model Adapter Layer (Multi-Model-Type Support)

CADENCE's reasoning core (CDAG, GNN, surrogate, PPO) operates on statistics over time and is model-agnostic. Only two responsibilities are model-type-specific, and each is isolated behind a thin adapter interface:

- Extracting an 'internal representation' proxy equivalent to a neural network's activations.
- Executing a 'partial retrain' appropriate to that model family.

Model Type	Internal-Representation Proxy	Partial-Retrain Mechanism
Neural network (MLP/CNN/small transformer)	Per-layer activation clusters (k-means over hidden-layer outputs)	Freeze all but the identified layer(s); EWC-regularized fine-tune
Tree ensemble (XGBoost/LightGBM)	Per-tree leaf-assignment distributions or SHAP contribution distributions	Additional boosting rounds targeted at the affected data region, or retraining only the trees that split on the drifted feature
Linear/logistic regression	Per-coefficient contribution distributions (weight x feature value)	Selectively refit specific coefficients, regularizing the rest toward prior values (an EWC-style penalty applied to a linear model)
Black-box / API-wrapped model	No internal nodes available; external feature-drift nodes plus output-distribution proxy only	Not supported — full retrain only, and the system communicates this limitation explicitly

Cross-adapter score comparability is not automatic: a responsibility score from the neural adapter and one from the tree adapter are not guaranteed to be on the same scale without an explicit calibration step. This is called out as an open design decision in Section 10.

6. Data Flow

The table below traces exactly what data moves between components, in pipeline order.

Stage	Data Passed
Production traffic to Collector	Feature vector, prediction, internal activations, delayed label (if/when available)
Collector to Drift Trigger	Rolling windowed statistics per feature and performance metric
Drift Trigger to CDAG Builder	Alert with the triggering window's identifier and current performance snapshot
CDAG Builder to GNN	Graph structure: node feature vectors (feature-node stats, activation-cluster stats) and edge weights
GNN to Surrogate	Per-node embedding vectors
Surrogate to PPO	Ranked responsibility scores per node
PPO to Executor	Chosen action (no-op / partial / full) and, for partial, the identified subnetwork
Executor to Validator	Candidate model checkpoint
Validator to Registry/Traffic	Promotion decision; on failure, a (predicted, actual) outcome pair fed back to the Surrogate's training set

7. Deployment View

7.1 What runs continuously (online)

- Inference Gateway and Streaming Collector — always on, low-latency path.
- Drift Detection Trigger — runs on a schedule (e.g., every N minutes) or per-window.

7.2 What runs on-demand (triggered by an alert)

- CDAG Builder, GNN inference, Surrogate inference, PPO inference — invoked only when a drift alert fires, since these are more compute-intensive.
- Retraining Executor — invoked only when PPO chooses partial or full retrain.

7.3 What is trained offline, ahead of time

- GNN + Surrogate joint training — trained periodically from an offline sandbox of drift-injection-and-fix experiments plus accumulated historical incidents.
- PPO policy training — trained in a simulated Gym-style environment before being trusted with live decisions; may be conservatively fine-tuned online later.

7.4 Hardware notes

The production model, GNN, and surrogate are all intentionally small enough to train and run on a single consumer GPU (4GB VRAM class) for the initial research build. PPO training happens against a simulator, not real infrastructure, so it does not require additional hardware beyond the local machine. Heavier

real-world datasets (e.g., full-scale Criteo/Avazu) are deferred to free cloud GPU sessions (Colab/Kaggle) rather than assumed as a hardware requirement.

8. Technology Stack

Layer	Technology
Languages	Python (core); optionally Go/Rust for a low-latency gateway
ML / DL	PyTorch, PyTorch Geometric (GNN), scikit-learn / LightGBM (surrogate), Stable-Baselines3 / RLlib (PPO)
Causal discovery	causal-learn, gcastle, or a custom NOTEARS implementation
Streaming / orchestration	Kafka or Redis Streams; Airflow / Prefect for retraining jobs
Serving	FastAPI / Triton Inference Server; Docker; Kubernetes (or single-node for the initial build)
Storage	PostgreSQL / TimescaleDB (metrics); MLflow (model registry)
Carbon / cost	CodeCarbon; Electricity Maps API
Monitoring	Grafana + Prometheus, or a custom dashboard
CI/CD	GitHub Actions

9. Non-Functional Considerations

Concern	Design Response
Latency	Inference Gateway path stays lightweight (prediction only); all attribution/retraining logic is off the hot path, triggered only on alert
Cost	PPO's reward explicitly penalizes compute cost, so unnecessary full retrains are actively discouraged
Carbon footprint	Carbon cost is a first-class term in the PPO reward, not an afterthought metric
Explainability	Every retraining decision is traceable to a specific CDAG node and responsibility score, logged in the audit log
Robustness	Shadow/canary validation prevents a bad fix from ever reaching full production traffic
Reproducibility	Docker images, versioned experiment configs (MLflow), and a public synthetic-benchmark generator with fixed seeds
Privacy	Activation sampling is privacy-filtered before leaving the Collector; no raw PII enters the CDAG

10. Open Design Decisions

These are the parts of the architecture that are intentionally not yet finalized. Each has a recommended default to unblock a first working build, but should be revisited or ablated once the pipeline runs end-to-end.

Open Decision	Recommended Default
Activation-to-node clustering method	K-means per layer, cluster count chosen via elbow method
GNN + Surrogate training regime	Train jointly, end-to-end, as one pipeline
PPO reward weights (compute / carbon / SLA)	Start with fixed weights, tune empirically against sandbox behavior
CADENCE's own drift-trigger mechanism	PSI + delayed-label F1 check (also used as the baseline to beat)
Shadow/canary promotion threshold	Promote if validation-window F1 is greater than or equal to the SLA target
Multi-node / multi-layer responsibility handling	Unfreeze every node/layer whose score exceeds a threshold tau;

Open Decision	Recommended Default
	treat very diffuse responsibility as a signal favoring full retrain
Cross-adapter score calibration	Not yet designed; required before claiming one unified dashboard across model types
Rolling window sizes for all statistics	To be tuned empirically per dataset during development

Appendix A: Glossary

Term	Meaning
CDAG	Causal Drift Attribution Graph — the core causal graph structure in CADENCE
Responsibility score	Predicted performance recovery if a given node's cause were fixed, minus current performance
EWC	Elastic Weight Consolidation — a regularization technique that protects important parameters during partial retraining
PPO	Proximal Policy Optimization — the reinforcement learning algorithm used for the Retraining Scope Optimizer
PSI	Population Stability Index — a statistical test for distribution shift
Shadow mode	Running a candidate model alongside production without using its predictions for real decisions, purely to validate it
Canary rollout	Gradually shifting real traffic to a new model, starting with a small percentage

Appendix B: Revision History

Version	Date	Change
1.0	Draft	Initial architecture document, consolidated from the CADENCE project proposal and design discussions